

Delta Lake

**Common Patterns.
Clear Fixes.**

50 engineering patterns covering Delta Lake internals and file layout, Z-order and liquid clustering, MERGE patterns, schema evolution, change data feed, OPTIMIZE and VACUUM, and deletion vectors in Databricks. Patterns every Delta Lake engineer must recognise.

INTRODUCTION

Delta Lake is deceptively simple. The problems are not.

Delta Lake looks simple from the outside. Write a `DataFrame`, read it back, merge updates. The API is familiar and the results look correct in development. The problems appear at scale, under concurrency, and after months of accumulation. A table that performed well at 10 million rows starts degrading at 500 million. `OPTIMIZE` runs that were supposed to speed things up make them slower. A `MERGE` that worked correctly in isolation creates duplicates when two jobs run simultaneously.

This book covers seven chapters. Chapter 1 covers Delta Lake internals and file layout on ADLS Gen2, the foundation that determines whether everything built on top of it performs well or degrades silently. Chapter 2 covers Z-order and liquid clustering and the specific conditions where each approach helps or hurts. Chapter 3 covers `MERGE` patterns and the failure modes that only appear at scale or under concurrency. Chapter 4 covers schema evolution and enforcement, and the ways automatic evolution can silently break Power BI and Microsoft Fabric consumers. Chapter 5 covers Change Data Feed and time travel, where Azure storage retention costs and consumer position tracking interact in ways engineers do not expect. Chapter 6 covers `OPTIMIZE` and `VACUUM` and the counterintuitive ways running maintenance too often or in the wrong order causes problems on ADLS Gen2. Chapter 7 covers deletion vectors and row tracking, where Delta protocol version requirements break older readers.

Every scenario follows the same structure: the situation, the wrong thinking, what is actually happening, and the fix. Delta Lake problems are particularly dangerous because they rarely raise errors. They manifest as degraded performance, wrong results, or unexpected costs. Recognising the pattern is the first step to avoiding it.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 2

Chapter 1

Delta Internals and File Layout

Eight scenarios covering how Delta Lake stores data on ADLS Gen2, why file layout determines query performance more than query logic, and the specific accumulation patterns that cause tables to degrade silently over time.

01

DELTA INTERNALS

Your Delta table has 500 million rows. Every query does a full scan.

THE SITUATION

A Delta table on ADLS Gen2 contains 500 million rows of event data partitioned by date. Queries filtering on a specific user ID or event type consistently scan the entire table regardless of the filter applied. Query times are measured in minutes even for filters that should return a small number of rows.

WRONG THINKING

Engineers assume partitioning is sufficient for query pruning and that filtering on any column will leverage file skipping. The table is partitioned by date so date filters are fast. Non-date filters are expected to benefit from Delta's file-level statistics.

WHAT IS ACTUALLY HAPPENING

Delta Lake uses file-level min and max statistics to skip files that cannot contain rows matching a filter. These statistics are stored per column per file in the transaction log. For file skipping to work effectively, the column being filtered must have a high correlation with the physical file layout. A date partition ensures all rows for a given date are in the same set of files. An unordered user ID or event type column is spread randomly across all files, so every file potentially contains any user ID. The min-max statistics for user ID in each file span nearly the full range of user IDs, meaning no files can be skipped. Every query must read every file to find the matching rows. On ADLS Gen2, reading every file in a large table is especially expensive because each file requires a separate authenticated HTTP request via the abfss driver.

THE FIX

Apply Z-order on columns that are frequently used as query filters but are not partition columns. Run `OPTIMIZE tablename ZORDER BY (user_id, event_type)` to co-locate rows with similar user IDs and event types within the same files. After Z-ordering, the min-max statistics for these columns will have much tighter ranges per file, enabling effective file skipping. For very large tables on Databricks Runtime 13 and above, evaluate liquid clustering as an alternative to Z-order which maintains clustering incrementally without requiring periodic full-table rewrites.

02

DELTA INTERNALS

You ran OPTIMIZE. Your queries got slower.

THE SITUATION

A Delta table with degraded query performance is identified. OPTIMIZE is run to compact small files and improve read performance. After OPTIMIZE completes the query times increase rather than decrease. The table now has fewer but larger files than before.

WRONG THINKING

Engineers run OPTIMIZE as a standard maintenance operation expecting it to always improve performance. OPTIMIZE is applied without analysing the current file layout or query patterns. Larger consolidated files are assumed to be universally better than many small files.

WHAT IS ACTUALLY HAPPENING

OPTIMIZE compacts small files into larger ones using the configured target file size. If the table has an effective partition structure and queries read specific partitions, OPTIMIZE within partitions is beneficial. However if OPTIMIZE is run without Z-order on a table where queries filter on non-partition columns, it consolidates random row orderings into larger files without improving data co-location. The resulting large files contain the same random mix of rows as the small files did, but now a query that previously skipped a small irrelevant file must read a large consolidated file that happens to contain a few matching rows mixed with many irrelevant ones. More data is scanned per matching row.

THE FIX

Always analyse query patterns before running OPTIMIZE. For tables where queries filter on specific columns, run OPTIMIZE with ZORDER BY on those columns rather than running OPTIMIZE alone. Plain OPTIMIZE without Z-order is most beneficial for tables where queries read full partitions or large sequential ranges and where file count reduction is the primary goal. Check query execution plans and file scan counts before and after OPTIMIZE to verify performance improved rather than assuming it did.

03

DELTA INTERNALS

Your Delta table has 10 million files. VACUUM takes longer than your ETL.

THE SITUATION

A Delta table on ADLS Gen2 that receives high-frequency small writes has accumulated 10 million files over several months. A VACUUM operation to clean up old files runs for 8 hours before timing out. The ETL pipeline that writes to the table runs in 2 hours. Maintenance is consuming more resources than the actual workload.

WRONG THINKING

Engineers allow files to accumulate over time assuming Delta Lake handles cleanup automatically. VACUUM is eventually run when storage costs become visible. The relationship between write frequency, file count, and maintenance overhead is not understood until the table reaches an unmanageable state.

WHAT IS ACTUALLY HAPPENING

Delta Lake does not automatically compact or delete files. Every write operation, regardless of size, creates new Parquet files. A streaming job or frequent small-batch job that writes every minute creates thousands of files per day. Without regular OPTIMIZE and VACUUM runs, file count grows continuously. At 10 million files the transaction log itself becomes very large, making every operation that reads the log slow. VACUUM at this scale must list and evaluate millions of files. On ADLS Gen2, each list operation is an authenticated HTTP request subject to the storage account's per-second request throughput limit (typically 20,000 requests per second per storage account). At 10 million files, listing alone can take hours and may trigger 503 throttling responses from the storage account that further slow the operation.

THE FIX

Implement regular OPTIMIZE and VACUUM as part of the table lifecycle from the first week of operation, not as a reactive measure after files accumulate. For tables that receive frequent small writes, run OPTIMIZE daily or after every significant write batch. Run VACUUM weekly with the default 7-day retention. For tables that have already accumulated millions of files, run OPTIMIZE in incremental partition-level batches rather than a full-table OPTIMIZE to avoid hitting timeout limits and ADLS Gen2 throttling. Consider enabling Auto Optimize on Databricks which applies incremental optimization automatically after writes. Spread very large tables across multiple storage accounts if a single account is near its request limit.

04

DELTA INTERNALS

You wrote 1000 rows to a Delta table. It created 200 files.

THE SITUATION

A pipeline writes 1000 rows to a Delta table using a Spark job with default parallelism. After the write the table contains 200 new Parquet files each containing 5 rows. Subsequent queries against the table are slow despite the small data volume.

WRONG THINKING

Engineers assume the number of output files is controlled by the data volume and that small writes produce few files. The relationship between Spark parallelism, shuffle partition count, and output file count is not understood. The write completes successfully so the output is assumed to be correct.

WHAT IS ACTUALLY HAPPENING

The number of Parquet files written by a Spark job equals the number of partitions in the DataFrame at write time. By default `spark.sql.shuffle.partitions` is set to 200. Even if the source data is a single file or a small in-memory collection, any transformation that triggers a shuffle produces 200 partitions. Writing a 200-partition DataFrame produces 200 output files regardless of how many rows each file contains. A table with thousands of 5-row files has extreme read overhead on ADLS Gen2 because Delta must open and read metadata for every file, and each file open is a separate authenticated request to the storage account.

THE FIX

Reduce output file count for small writes by calling `coalesce()` or `repartition()` before writing. For a 1000-row write, `df.coalesce(1).write` or `df.repartition(4).write` produces 1 to 4 files rather than 200. Set `spark.sql.shuffle.partitions` to a value appropriate for the data volume rather than leaving the default of 200 for all workloads. Enable Auto Optimize on Databricks which automatically coalesces small files after writes using optimized writes, reducing file count without requiring manual `coalesce()` calls.

05

DELTA INTERNALS

Your Delta table reads are fast. Writes are getting slower every week.

THE SITUATION

A Delta table that initially had fast write performance has degraded over months. Read performance remains good. Write operations that previously completed in 5 minutes now take 25 minutes for the same data volume. Nothing in the write logic has changed.

WRONG THINKING

Engineers investigate the write pipeline code and Spark configuration looking for performance regressions. Since reads are still fast the table itself appears healthy. The transaction log is not considered as a source of write overhead.

WHAT IS ACTUALLY HAPPENING

Delta Lake maintains a transaction log that records every write operation as a JSON file in the `_delta_log` directory. Before writing, Spark must read the transaction log to determine the current table state and check for conflicts. As the number of transactions grows, reading the full transaction log becomes increasingly expensive. Delta Lake creates checkpoint files every 10 transactions to compact the log, but even with checkpointing, a table with thousands of transactions has more log metadata to process than a table with hundreds. Write performance degrades as transaction history accumulates because every write operation must verify the current state against the growing log. On ADLS Gen2 the cost of reading transaction log files is amplified by per-request latency, so log growth has a larger impact on write performance than it would on a local file system.

THE FIX

Run `OPTIMIZE` regularly to compact data files and trigger log checkpointing. Monitor the number of files in the `_delta_log` directory and the size of the transaction log. If the log has grown to thousands of JSON files between checkpoints, the checkpointing interval may need adjustment. For tables with very high write frequency, consider partitioning to reduce the scope of conflict checking. In Databricks, enabling optimized writes reduces the number of small transactions by batching write operations, which reduces log growth rate and maintains write performance over time.

06

DELTA INTERNALS

You cloned a Delta table. The clone is 3x larger than the original.

THE SITUATION

A Delta table of 50GB is cloned using `CREATE TABLE CLONE` from one ADLS Gen2 container to another. The clone appears in storage consuming 150GB. The original table is 50GB. The total storage cost for the clone operation is 3 times the expected amount and the clone copy operation also generated cross-container data transfer charges within the storage account.

WRONG THINKING

Engineers assume a Delta clone is a lightweight metadata operation that does not duplicate data. Cloning is understood to be similar to a snapshot or pointer operation. The distinction between shallow clone and deep clone and their storage implications is not understood.

WHAT IS ACTUALLY HAPPENING

Delta Lake supports two clone types. A shallow clone creates a copy of the table metadata and transaction log but references the original table's data files without copying them. Storage cost is minimal. A deep clone copies both the metadata and all the underlying Parquet data files to the new location. A deep clone of a 50GB table creates 50GB of new data files on ADLS Gen2. If the table has been written to multiple times without running `VACUUM`, the transaction log references both current files and older versions that have not been vacuumed. A deep clone copies all referenced files including historical versions, which is why the clone can be larger than the apparent current table size. If the clone destination is in a different region or a different storage account, additional Azure egress charges apply.

THE FIX

Use shallow clone for development, testing, and read-only reference use cases where data file duplication is not needed. Use deep clone only when data portability and independence from the source table are required. Before deep cloning, run `VACUUM` on the source table with an appropriate retention period to remove historical files that do not need to be included in the clone. Check the size of the source table's data files before cloning by inspecting the file count and sizes in the table detail output of `DESCRIBE DETAIL tablename`. Verify that the clone destination is in the same Azure region as the source to avoid cross-region egress charges.

07

DELTA INTERNALS

Your partition column has 10000 distinct values. Performance is terrible.

THE SITUATION

A Delta table on ADLS Gen2 is partitioned on a customer ID column that has 10,000 distinct values. The intention was to improve query performance for customer-specific queries. Instead, queries that filter on customer ID are slower than on the unpartitioned version of the same table, and listing the table directory takes minutes.

WRONG THINKING

Engineers apply Delta Lake partitioning following the general principle that partitioning improves query performance by enabling partition pruning. Customer ID is a frequently used filter column so partitioning on it seems logical. The effect of high-cardinality partitioning on ADLS Gen2 metadata operations is not considered.

WHAT IS ACTUALLY HAPPENING

Delta Lake partitioning creates a separate directory for each distinct partition value. A table with 10,000 distinct customer IDs creates 10,000 directories in the storage layer. On ADLS Gen2 with hierarchical namespace enabled, each directory is a real filesystem object. Listing all partition directories requires 10,000 list API calls, each subject to storage account request limits. Writing to any partition requires checking the partition directory structure, and the transaction log must track file paths that include the partition directory for every file. For queries that access many customers simultaneously, the partition directory traversal across thousands of paths is slower than a full table scan on a well-maintained unpartitioned table with Z-order. Storage account throttling can also kick in if a job lists many partitions concurrently.

THE FIX

Partition Delta tables only on low-cardinality columns with fewer than a few hundred distinct values. Date, region, status, and product category are appropriate partition columns. High-cardinality identifiers like customer ID, user ID, and transaction ID should never be partition columns. For high-cardinality filter columns, use Z-order or liquid clustering to improve file skipping without creating directory explosion. As a guideline, if a partition column has more than 1000 distinct values it is almost certainly the wrong partition choice on ADLS Gen2.

08

DELTA INTERNALS

You deleted 1 row from a Delta table. 50 files were rewritten.

THE SITUATION

A single-row delete is executed against a Delta table using `DELETE FROM tablename WHERE id = 12345`. The operation completes successfully but the cluster shows significant write activity. Examining the transaction log reveals that 50 Parquet files were rewritten as part of the delete operation.

WRONG THINKING

Engineers assume a single-row delete is a lightweight operation. Deleting one row should involve minimal I/O. The copy-on-write nature of Delta Lake deletes is not understood.

WHAT IS ACTUALLY HAPPENING

Delta Lake uses copy-on-write semantics for delete operations by default. When a row is deleted, Delta cannot modify the existing Parquet file in place because Parquet files are immutable. Instead Delta must rewrite every file that contains a row matching the delete condition. If the deleted row ID is distributed across 50 files due to random insertion order or if the table has no file skipping capability for the ID column, all 50 files must be read, the matching row removed from each, and the file rewritten. A single-row delete on a poorly laid out table can trigger a rewrite of a significant fraction of the table's data files. On ADLS Gen2 this means 50 read operations followed by 50 write operations, with the associated per-request costs and latencies.

THE FIX

Apply Z-order on the column used in delete conditions before running large-scale delete operations. When rows with similar IDs are co-located in fewer files, a delete affecting one ID touches only one or two files rather than hundreds. For tables on Databricks Runtime 12.2 and above, enable deletion vectors which mark deleted rows in a separate metadata file without rewriting the underlying Parquet file. Deletion vectors make individual row deletes nearly instantaneous by avoiding file rewrites entirely. Enable them with `ALTER TABLE tablename SET TBLPROPERTIES (delta.enableDeletionVectors = true)`.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 2

Chapter 2

Z-Order and Liquid Clustering

Seven scenarios covering when Z-order helps, when it hurts, and the specific conditions where liquid clustering is the better choice and where it is not.

09

Z-ORDER AND LIQUID CLUSTERING

You Z-ordered on 5 columns. Performance did not improve.

THE SITUATION

Z-order is applied to a Delta table specifying 5 columns commonly used in query filters. After Z-ordering, query performance shows no measurable improvement. The OPTIMIZE with ZORDER command completed successfully.

WRONG THINKING

Engineers assume that Z-ordering on more columns provides better performance because more query patterns can benefit from file skipping. Adding all commonly queried columns to the Z-order specification seems thorough.

WHAT IS ACTUALLY HAPPENING

Z-order is a space-filling curve that maps multi-dimensional data into a single dimension while preserving locality. It works best when applied to 1 to 3 columns. With 5 or more columns, the Z-order algorithm must distribute data locality across too many dimensions. The result is that no single dimension has strong locality, and file skipping becomes ineffective for any individual column filter. The effectiveness of Z-order degrades exponentially as the number of Z-order columns increases. Five-column Z-order often performs worse than two-column Z-order for the most frequent query patterns.

THE FIX

Limit Z-order to the 1 to 3 columns most frequently used as query filters. Identify the top filter columns by analysing query patterns in the query history rather than specifying all possible filter columns. If more than 3 columns need effective file skipping, evaluate liquid clustering which uses a different algorithm that handles multiple clustering columns more effectively. For tables with 4 or more frequently queried non-partition columns, liquid clustering on Databricks Runtime 13 and above will typically outperform Z-order.

10

Z-ORDER AND LIQUID CLUSTERING

Your Z-order takes 4 hours to run. It needs to run daily.

THE SITUATION

A daily OPTIMIZE with ZORDER BY job was set up to maintain query performance on a large Delta table on ADLS Gen2. The Z-order job is taking 4 hours to complete. The total daily maintenance window is 3 hours. The Z-order cannot complete within the available time and is being cancelled before finishing.

WRONG THINKING

Engineers implement Z-order as a standard maintenance operation and schedule it daily following best practice guidance. The time required to Z-order a large table is not estimated before scheduling. The assumption is that Z-order is a metadata operation that runs quickly.

WHAT IS ACTUALLY HAPPENING

Z-order rewrites data files to physically co-locate rows with similar column values. For a large table, this requires reading all data files from ADLS Gen2, sorting and reorganising the data according to the Z-order curve, and writing new Parquet files. It is a full data rewrite operation. The time required scales with total data volume, not with the amount of data that changed since the last Z-order run. Running Z-order daily on a 10TB table performs a full 10TB rewrite every day regardless of whether only 1GB of new data was added. The operation is not incremental. On Azure, this rewrite also generates significant storage account read and write traffic which can compete with concurrent workloads for the storage account's request throughput.

THE FIX

Run OPTIMIZE with ZORDER BY less frequently for large tables. For tables receiving daily incremental loads, Z-ordering the entire table daily is wasteful. Consider running Z-order only on recently modified partitions by targeting specific partition values in the OPTIMIZE command. Alternatively evaluate liquid clustering which is designed to be incremental. Liquid clustering applies clustering progressively to files that need it without rewriting the entire table, making it far more efficient for tables with frequent incremental updates.

11

Z-ORDER AND LIQUID CLUSTERING

You switched from Z-order to liquid clustering. Performance got worse.

THE SITUATION

A table using Z-order with good query performance is migrated to liquid clustering following a recommendation to use the newer feature. After enabling liquid clustering and running the initial clustering operation, query performance degrades compared to the previous Z-order baseline.

WRONG THINKING

Engineers treat liquid clustering as a direct drop-in replacement for Z-order that is always better. Since liquid clustering is the newer approach it is assumed to be superior in all cases. The migration is done without benchmarking.

WHAT IS ACTUALLY HAPPENING

Liquid clustering applies clustering incrementally and progressively. When first enabled on an existing table, only newly written data and data processed by explicit OPTIMIZE runs are clustered. Existing files retain their previous layout. If the initial OPTIMIZE after enabling liquid clustering does not fully process all existing files, a large portion of the table remains unclustered. Queries that previously benefited from thorough Z-order coverage now read a mix of clustered new files and unclustered old files. The transition period can produce worse performance than either a fully Z-ordered table or a fully liquid-clustered table.

THE FIX

After enabling liquid clustering on an existing table, run a full OPTIMIZE to cluster all existing files before benchmarking performance. Run `OPTIMIZE tablename` without additional arguments after setting the cluster by columns. Monitor the clustering state using `DESCRIBE DETAIL tablename` and check the numFilesInClusteringJob metric to confirm clustering has been applied to the majority of files. Only benchmark liquid clustering performance after the initial full clustering pass is complete.

12

Z-ORDER AND LIQUID CLUSTERING

Your liquid clustering is defined. No clustering has actually happened.

THE SITUATION

Liquid clustering is enabled on a new Delta table by defining cluster by columns in the table creation statement. Data is written to the table daily. After two weeks of ingestion, query performance is not improving as expected. Inspecting the table details shows that clustering has not been applied to most of the data.

WRONG THINKING

Engineers assume enabling liquid clustering in the table definition automatically clusters data as it is written. Since the cluster by columns are defined at table creation, all subsequent writes are expected to be clustered automatically.

WHAT IS ACTUALLY HAPPENING

Liquid clustering does not cluster data automatically during writes. Defining cluster by columns specifies the desired clustering layout but does not trigger clustering operations. Data written to a liquid clustering table is written in the order it arrives, which may not match the clustering layout. Actual clustering is applied by running the OPTIMIZE command explicitly or by enabling Auto Liquid Clustering which triggers incremental clustering automatically. Without OPTIMIZE runs or auto clustering, the table accumulates unclustered data regardless of how the cluster by columns are configured.

THE FIX

After enabling liquid clustering, implement a regular OPTIMIZE schedule to apply clustering to newly written data. Run `OPTIMIZE tablename` after each significant write batch or on a daily schedule. Enable Auto Liquid Clustering in Databricks by setting the table property `delta.autoOptimize.autoCompact = true` alongside liquid clustering to have Databricks automatically apply incremental clustering after writes. Verify clustering is being applied by checking the `DESCRIBE DETAIL` output for clustering metrics after the first OPTIMIZE run.

13

Z-ORDER AND LIQUID CLUSTERING

Z-order improved performance for one query pattern. Made everything else slower.

THE SITUATION

A Delta table is Z-ordered on the most frequent query filter column, improving that query pattern by 80 percent. A week later, engineers from another team report that their queries, which filter on a different column, have gotten significantly slower since the Z-order was applied.

WRONG THINKING

Engineers apply Z-order based on their team's query patterns without considering other consumers of the table. Since Z-order is a read optimisation it is assumed to be beneficial for all readers. The physical data reorganisation that Z-order applies is not understood to have trade-offs for different access patterns.

WHAT IS ACTUALLY HAPPENING

Z-order physically reorganises data files to co-locate rows with similar values in the Z-ordered columns. This improves file skipping for queries that filter on those columns but degrades file skipping for queries that filter on other columns. Before Z-ordering, data files may have had a reasonable distribution of values for multiple columns. After Z-ordering on column A, files are tightly grouped by column A values but column B values are now randomly distributed across files again. A query filtering on column B that previously had some file skipping ability now has less because Z-order disrupted whatever natural distribution column B had.

THE FIX

Coordinate Z-order decisions across all teams consuming a shared table. Before applying Z-order, identify all active query patterns against the table and evaluate whether the proposed Z-order columns serve the majority of query traffic. If two teams have conflicting query patterns, consider whether the table should be split into separate tables optimised for each access pattern, or whether materialized views or Gold layer tables tailored to each team's needs are a better solution than trying to satisfy all patterns with a single Z-order.

14

Z-ORDER AND LIQUID CLUSTERING

You Z-ordered a table that receives daily appends. The optimization resets every run.

THE SITUATION

A Delta table receives daily append operations adding new data. OPTIMIZE with ZORDER BY is run after each daily append to maintain clustering. Each OPTIMIZE run takes several hours because it rewrites the entire table. The Z-order applied yesterday is partially undone by today's unordered append and then requires another full rewrite to restore.

WRONG THINKING

Engineers run full Z-order maintenance after every write to maintain data layout quality. The assumption is that Z-order must be reapplied after every write because new data is not clustered. The cumulative cost of daily full-table rewrites is not calculated.

WHAT IS ACTUALLY HAPPENING

OPTIMIZE with ZORDER BY rewrites files to improve data co-location. When new unordered data is appended to a Z-ordered table, the new files are not Z-ordered but existing files retain their Z-order layout. Running OPTIMIZE again rewrites both the existing files and the new files to produce a globally Z-ordered result. For daily appends that add a small fraction of the total table size, this means rewriting the entire table daily to re-integrate the small amount of new data. The maintenance cost grows linearly with total table size regardless of daily incremental volume. On Azure, this also generates continuous high-volume ADLS Gen2 traffic which contributes to storage account request rates.

THE FIX

For tables with frequent incremental appends, use liquid clustering instead of Z-order. Liquid clustering is incremental by design. It clusters only the files that need clustering, which for daily appends means only the newly written files plus any unclustered older files. The daily maintenance cost is proportional to the amount of new data, not the total table size. For large tables with daily incremental loads, switching from Z-order to liquid clustering can reduce daily maintenance time from hours to minutes.

15

Z-ORDER AND LIQUID CLUSTERING

Your liquid clustering column has too many distinct values. No benefit observed.

THE SITUATION

Liquid clustering is configured on a Delta table using a transaction timestamp column with millisecond precision. The timestamp has billions of distinct values. After running OPTIMIZE to apply clustering, query performance shows no improvement for queries filtering on time ranges.

WRONG THINKING

Engineers choose the timestamp column for liquid clustering because time range queries are the most common access pattern. A column with many distinct values seems like a good clustering key because it provides fine-grained sorting. The relationship between cardinality, file statistics, and clustering effectiveness is not considered.

WHAT IS ACTUALLY HAPPENING

Liquid clustering and Z-order both work by co-locating rows with similar column values within the same files, enabling file-level min-max statistics to skip files that cannot contain matching rows. For this to work, files must contain rows from a narrow range of column values. A millisecond-precision timestamp with billions of distinct values naturally has a wide value range per file even after clustering because the clustering algorithm can only consolidate rows into a finite number of files. Clustering on a high-cardinality timestamp with narrow query windows such as 5-minute ranges produces files that each contain a wide time range, providing limited file skipping for narrow window queries.

THE FIX

For time-range queries, partition the table by date or hour rather than relying on clustering. Date or hour partitioning ensures all rows for a given time period are in dedicated files, providing complete partition pruning for time-range filters. Use liquid clustering or Z-order on secondary filter columns within partitions rather than on the primary time column. For sub-day time ranges, a partition by hour combined with a secondary Z-order on the query's secondary filter column provides more effective skipping than clustering on the raw timestamp alone.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 2

Chapter 3

MERGE Patterns

Ten scenarios covering MERGE performance at scale, the concurrency and correctness failures that appear under load, and the specific ways MERGE semantics differ from what engineers expect.

16

MERGE PATTERNS

Your MERGE runs correctly on 1 million rows. Times out on 100 million.

THE SITUATION

A MERGE operation that upserts incoming data into a target Delta table works correctly and quickly in development with 1 million rows. When promoted to production where the target table has 100 million rows, the MERGE times out after 2 hours.

WRONG THINKING

Engineers test MERGE at a smaller data scale and confirm correct behavior. Assuming linear scaling, a 100x larger dataset is expected to take proportionally longer but still complete within acceptable time. The non-linear scaling of MERGE with table size is not anticipated.

WHAT IS ACTUALLY HAPPENING

MERGE in Delta Lake requires the Spark engine to join the source data against the target table to identify matching rows. For a target table with 100 million rows and no effective data skipping, this join requires scanning the full target table for every batch of source rows. The join cost scales with the product of source size and target size rather than linearly with either alone. Additionally each matched row triggers a file rewrite due to copy-on-write semantics. As the target table grows, both the join cost and the rewrite cost increase significantly, producing superlinear scaling that makes a MERGE that ran in minutes at small scale take hours at large scale.

THE FIX

Apply Z-order or liquid clustering on the MERGE join key column before running large-scale MERGEs. Effective file skipping means the engine can identify which target files potentially contain matching rows without scanning the entire table. Use partition filtering in the MERGE condition to limit the target table scan to relevant partitions. Add a partition filter in the WHEN clause such as filtering on a date range that covers the source data. For very large target tables, consider breaking the MERGE into smaller batches partitioned by date or ID range rather than merging all source data in a single operation.

17

MERGE PATTERNS

Your MERGE is creating duplicate rows despite a unique match condition.

THE SITUATION

A MERGE operation specifies a match condition on a primary key column that is unique in the target table. After several MERGE runs, the target table contains duplicate rows for some key values. The MERGE logic appears correct when inspected.

WRONG THINKING

Engineers assume a MERGE with a unique key match condition cannot produce duplicates because the uniqueness of the key guarantees at most one matching row per source row. The possibility of duplicates in the source data feeding the MERGE is not checked.

WHAT IS ACTUALLY HAPPENING

Delta Lake MERGE does not validate that the source data contains unique values for the match key. If the source DataFrame has two rows with the same key value, both rows match the same target row. The MERGE behavior with multiple source rows matching one target row is undefined in the SQL standard and varies by implementation. In Spark's implementation it can result in the target row being updated multiple times or new rows being inserted, leading to duplicates in the target. The uniqueness constraint applies to the target table but there is no built-in enforcement that source data is also deduplicated before MERGE.

THE FIX

Deduplicate the source DataFrame on the MERGE key before executing the MERGE. Use

`dropDuplicates([key_column])` or a window function with `row_number()` to keep only one record per key in the source. Add a data quality check that asserts source key uniqueness before the MERGE and fails the pipeline with a clear error if duplicates are found rather than allowing them to silently corrupt the target. Never assume upstream data quality without explicit validation.

18

MERGE PATTERNS

MERGE performance degrades every week as the target table grows.

THE SITUATION

A MERGE operation that initially ran in 10 minutes now runs in 90 minutes after 3 months of daily operation. The source data volume is identical to when the MERGE was first deployed. Nothing in the pipeline logic has changed. Only the target table has grown.

WRONG THINKING

Engineers investigate the pipeline code and source data for regressions when MERGE performance degrades. Since the pipeline logic has not changed, the cause is assumed to be infrastructure or Databricks platform related. The target table size as a factor in MERGE performance is not considered.

WHAT IS ACTUALLY HAPPENING

MERGE performance is directly affected by target table size when file skipping is not effective. As the target table grows through daily appends and updates, the number of files the engine must scan to find matching rows increases. If Z-order or partition pruning is not effective for the MERGE join key, every MERGE scans a growing fraction of the target table. A 10-minute MERGE on a 1GB table becomes a 90-minute MERGE on a 9GB table because the scan cost scales with table size. Regular OPTIMIZE without ZORDER does not help because it consolidates files without improving data co-location for the join key.

THE FIX

Apply OPTIMIZE with ZORDER BY on the MERGE join key column and maintain it regularly. Monitor target table size and MERGE execution time together to identify when performance begins degrading. Implement partition-based MERGE where the MERGE condition includes a partition filter that restricts the target table scan to partitions that are relevant to the current source batch. For SCD Type 2 patterns where the target table grows indefinitely, partition by a time-based column and include a partition filter in the MERGE to limit the scan to active records.

19

MERGE PATTERNS

Your MERGE rewrites the entire target table even when only 100 rows matched.

THE SITUATION

A MERGE operation processes 100 matching rows out of a target table with 10 million rows. After the MERGE, the transaction log shows that thousands of files were rewritten. Storage costs on ADLS Gen2 from the MERGE are significant relative to the small number of rows that actually changed.

WRONG THINKING

Engineers assume MERGE rewrites only the files containing matched rows. Since only 100 rows matched, only a handful of files should be rewritten. The copy-on-write semantics of Delta and the relationship between file rewrite count and data distribution are not understood.

WHAT IS ACTUALLY HAPPENING

Delta Lake MERGE uses copy-on-write semantics. For every file that contains at least one matched row, the entire file is read, modified, and rewritten as a new Parquet file. The number of files rewritten depends on how widely the 100 matched rows are distributed across the target table's files. If the target table has no Z-order or clustering on the join key, the 100 matched rows may be spread across hundreds or thousands of files because they were inserted in random order over time. Each of those files must be fully rewritten to incorporate the change to one or two rows within it. The result is disproportionate write amplification and significant ADLS Gen2 read and write traffic relative to the size of the logical change.

THE FIX

Co-locate rows that are likely to be updated together by applying Z-order or liquid clustering on the MERGE join key. When matched rows are concentrated in fewer files, the rewrite amplification is proportionally lower. For tables where rows are frequently updated, enable deletion vectors which allow row-level deletes and updates to be marked in metadata without rewriting the entire file. Deletion vectors reduce write amplification significantly for update-heavy workloads. Enable with `ALTER TABLE tablename SET TBLPROPERTIES (delta.enableDeletionVectors = true)`.

20

MERGE PATTERNS

You used MERGE for SCD Type 2. End dating logic is missing rows.

THE SITUATION

A MERGE-based SCD Type 2 implementation closes active records by setting an end date and inserts new records for changed values. After several months of operation, audit queries discover that some historical records do not have correct end dates. Some active records that should have been closed are still showing as active.

WRONG THINKING

Engineers implement SCD Type 2 using MERGE and test with small datasets. The logic produces correct results in testing. The assumption is that MERGE handles all match cases correctly for SCD Type 2 patterns. Edge cases with multiple updates to the same key in a single source batch are not tested.

WHAT IS ACTUALLY HAPPENING

SCD Type 2 MERGE logic typically closes the existing active record and inserts a new active record in the same operation. When the source batch contains multiple changes for the same key within the same batch, the MERGE logic may only process one of the changes per key depending on how the source is structured. If two updates to the same customer ID arrive in the same batch, the MERGE may match the first update correctly but the second update has no active record to close because the first update already closed it. The second record may be inserted as a new active record without closing the intermediate version that the first update created.

THE FIX

Deduplicate source data before SCD Type 2 MERGE to ensure at most one change per key per batch. Process multiple changes to the same key sequentially across separate batches rather than handling them in a single MERGE. For high-frequency change data that may include multiple updates per key per batch, implement a pre-MERGE step that orders and deduplicates changes per key, retaining only the latest change for each key in the current batch. Add audit queries that validate SCD Type 2 invariants after each MERGE run, specifically checking for duplicate active records or records with missing end dates.

21

MERGE PATTERNS

Your MERGE works in isolation. Fails when run concurrently by two jobs.

THE SITUATION

Two pipeline jobs both run MERGE operations against the same Delta target table. When run sequentially each job succeeds. When both jobs run concurrently one of them fails with a concurrent modification exception or produces incorrect results.

WRONG THINKING

Engineers test each pipeline job in isolation and confirm correct behavior. Concurrent execution is not tested before going to production because the two jobs process different source data. The assumption is that since they process different data they will not conflict.

WHAT IS ACTUALLY HAPPENING

Delta Lake uses optimistic concurrency control. When two MERGE operations run simultaneously, each reads the current table state at the start of the operation and attempts to commit its changes at the end. Delta detects when two concurrent operations have modified the same files and rejects the second commit with a concurrent modification error. Even if the two jobs update different rows, if those rows are stored in the same files, Delta treats this as a conflict. For MERGE operations which can touch many files, concurrent conflicts are common even when the logical data being updated is completely separate.

THE FIX

Serialise MERGE operations against the same target table when possible. If both jobs must run concurrently, partition the target table such that each job exclusively writes to separate partitions, which Delta can commit without conflict. Use partition-level MERGE filters to ensure each job only touches its designated partitions. Implement retry logic for the concurrent modification exception since the failed job can simply retry and succeed once the other job has committed. Enable Delta Lake auto retry on transient conflicts using Databricks Workflows retry configuration.

22

MERGE PATTERNS

You replaced MERGE with INSERT OVERWRITE. Data loss appeared on failure.

THE SITUATION

To improve MERGE performance, a team switches to INSERT OVERWRITE for a daily batch that was previously using MERGE. Performance improves significantly. On the first pipeline failure after the switch, the overwrite partially completed before failing and the target partition is left empty. Recovery requires reprocessing the source data.

WRONG THINKING

Engineers replace MERGE with INSERT OVERWRITE as a performance optimisation after finding that MERGE was too slow. INSERT OVERWRITE is simpler and faster. The atomicity difference between MERGE and INSERT OVERWRITE is not evaluated before the change is made.

WHAT IS ACTUALLY HAPPENING

Delta Lake MERGE operations are atomic. If a MERGE fails partway through, the transaction is rolled back and the target table is unchanged. INSERT OVERWRITE in Delta Lake is also atomic per write, but its failure recovery characteristics differ from MERGE. When INSERT OVERWRITE targets a specific partition and fails mid-write, Delta rolls back the write leaving the original partition data intact. However if the pipeline framework treats the overwrite as a multi-step operation or if the partition was explicitly deleted before the overwrite, a failure can leave the partition empty or in an inconsistent state depending on how the pipeline manages the overwrite transaction.

THE FIX

Ensure that INSERT OVERWRITE operations use Delta Lake's transactional semantics correctly. Always use the full INSERT OVERWRITE INTO tablename SELECT statement within a single transaction rather than deleting the partition first and then inserting. Verify that pipeline failure recovery restores the partition to its pre-overwrite state by testing explicit failure scenarios before deploying. For critical data where data loss on failure is unacceptable, MERGE's atomic update semantics are worth the performance trade-off. Document the failure behavior of each write pattern explicitly in pipeline design decisions.

23

MERGE PATTERNS

Your MERGE source has duplicates. The target now has duplicates too.

THE SITUATION

A MERGE operation runs successfully and inserts new records into the target table. A data quality check after the MERGE finds duplicate rows in the target for some primary key values. The MERGE logic includes a WHEN NOT MATCHED THEN INSERT clause that inserts source rows that have no match in the target.

WRONG THINKING

Engineers assume a MERGE with a WHEN NOT MATCHED INSERT clause inserts each unmatched source row exactly once. Since the target table did not previously contain the key, the insert should produce exactly one row per new key. Duplicates in the source data for new keys are not anticipated.

WHAT IS ACTUALLY HAPPENING

WHEN NOT MATCHED THEN INSERT inserts all source rows that have no corresponding match in the target. If the source contains two rows with the same key value and neither matches an existing target row, both rows are inserted as new rows. Delta Lake does not enforce uniqueness constraints on insert. The MERGE does not deduplicate unmatched source rows before inserting them. This is the same root cause as scenario 17 but specifically in the INSERT path rather than the UPDATE path. The target ends up with two rows for a key that should have exactly one.

THE FIX

Apply source deduplication before all MERGE operations regardless of whether the operation is primarily an update or an insert pattern. Use `dropDuplicates([key_column])` on the source DataFrame before passing it to the MERGE. For CDC-based MERGE patterns where duplicate records can arrive from the source system, implement a pre-MERGE deduplication step that retains the most recent record per key based on a timestamp or sequence number column. Treat source data quality as a MERGE precondition and fail early if the precondition is violated.

24

MERGE PATTERNS

MERGE performance degrades on tables with many small partitions.

THE SITUATION

A Delta table on ADLS Gen2 is partitioned on a date column with daily partitions. After two years of daily operation, the table has 730 date partitions each containing many small files. A daily MERGE that adds and updates records is taking progressively longer and the transaction log operations before the MERGE starts are themselves taking several minutes.

WRONG THINKING

Engineers assume that date partitioning helps MERGE performance by limiting the scope of the target table scan. More partitions should mean better isolation and faster operations. The overhead of managing hundreds of partitions in the transaction log and on ADLS Gen2 is not considered.

WHAT IS ACTUALLY HAPPENING

Delta Lake's transaction log tracks metadata for every file in every partition. A table with 730 partitions each containing 50 files has 36,500 file metadata entries in the transaction log. Before a MERGE can begin, Spark reads the transaction log to determine the current table state. Reading and parsing the metadata for 36,500 files takes measurable time even before any data is scanned. As partition count grows with each new day, this metadata overhead grows linearly. Additionally listing all partition directories for partition pruning involves many file system API calls. On ADLS Gen2 with hierarchical namespace, each directory listing is an authenticated HTTP request to the storage account, and the cumulative latency of listing 730 directories adds noticeable startup time to every MERGE.

THE FIX

Run OPTIMIZE regularly to reduce file count within each partition, which reduces the number of metadata entries in the transaction log. For old partitions that are no longer updated, compact them to a minimal number of large files and VACUUM old file versions to reduce the transaction log footprint. Consider whether daily partitioning is necessary or whether weekly or monthly partitioning would provide sufficient partition pruning while reducing partition count. Enable Delta log checkpointing to ensure the transaction log is regularly compacted and fast to read.

25

MERGE PATTERNS

Your MERGE used to take 10 minutes. After adding a new column it takes 2 hours.

THE SITUATION

A schema change adds a new column to a Delta target table. A MERGE pipeline that previously ran in 10 minutes now takes 2 hours for the same source data volume. No other changes were made to the pipeline or the table.

WRONG THINKING

Engineers assume adding a column to a Delta table is a lightweight metadata operation with no impact on existing query performance. Since the column is new and nullable, existing data is not changed. MERGE performance degradation after a schema change is not anticipated.

WHAT IS ACTUALLY HAPPENING

Adding a new column to a Delta table with existing data can invalidate the file-level column statistics used for file skipping. Delta stores per-column min-max statistics for each file to enable predicate pushdown and file skipping. When a new column is added, existing files do not contain statistics for that column because it did not exist when those files were written. If the MERGE condition or any filters reference the new column, Delta cannot use file skipping for any existing files because the statistics are absent. The engine must read all existing files to evaluate the filter, eliminating the file skipping benefit that was responsible for the previous 10-minute runtime.

THE FIX

After adding a new column, run OPTIMIZE to rewrite existing files with statistics that include the new column. This allows Delta to generate file-level statistics for the new column in the rewritten files, restoring file skipping capability. For tables where MERGE performance is critical, run a full OPTIMIZE with ZORDER BY on the MERGE key columns after any schema change that adds columns used in MERGE conditions or query filters. Monitor file skipping metrics in the query execution plan before and after schema changes to confirm skipping is functioning as expected.

04

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 2

Chapter 4 Schema Evolution and Enforcement

Seven scenarios covering how Delta Lake handles schema changes, the ways automatic schema evolution can silently break downstream Power BI and Microsoft Fabric consumers, and when schema enforcement causes unexpected pipeline failures.

26

SCHEMA EVOLUTION

You enabled schema evolution. A source added a column. Downstream broke.

THE SITUATION

Schema evolution is enabled on a Delta table using `mergeSchema = true`. A source system adds a new column to its output. The pipeline writes the new column into the Delta table successfully. A downstream Power BI semantic model and several SQL queries that use `SELECT *` now return different column counts and break existing logic that relied on a fixed schema.

WRONG THINKING

Engineers enable schema evolution to avoid pipeline failures when source schemas change. Automatic schema evolution feels like the safer option because it prevents write failures. The downstream impact of adding columns to a table that consumers depend on is not considered.

WHAT IS ACTUALLY HAPPENING

Schema evolution allows new columns to be added to a Delta table automatically when the source data contains columns not present in the existing schema. This is convenient for the writer but it is a breaking change for any downstream consumer that relies on column count, column position, or a fixed schema. A Power BI semantic model that imports the table schema from a Databricks SQL warehouse, a SQL query using `SELECT *`, or a downstream pipeline that unpacks the DataFrame by position will all behave unexpectedly when a new column appears. Schema evolution solves the write problem by creating a downstream compatibility problem.

THE FIX

Treat schema evolution as a deliberate schema change that requires downstream impact assessment, not an automatic convenience feature. Before enabling `mergeSchema`, identify all downstream consumers of the table and evaluate whether they can handle new columns. For critical serving tables consumed by Power BI or Fabric, disable schema evolution and handle schema changes explicitly through a controlled deployment process. When schema evolution is appropriate, notify downstream consumers before the change takes effect so they can update their logic to handle the new column.

27

SCHEMA EVOLUTION

Your schema evolution silently changed a column type from int to long.

THE SITUATION

A source system changes a numeric column from 32-bit integer to 64-bit long. Schema evolution automatically widens the column type in the Delta table. Downstream consumers that read the column and cast it to int begin failing with overflow errors or truncation. No schema change alert was raised.

WRONG THINKING

Engineers assume schema evolution only adds columns and that existing columns are never modified. Type changes in the source are not anticipated as something that schema evolution would propagate automatically. The downstream impact of a widened numeric type is not considered.

WHAT IS ACTUALLY HAPPENING

Delta Lake schema evolution supports type widening for safe promotions such as int to long, float to double, and byte to short. When a source writes a long value to a column previously defined as int, Delta automatically widens the column type rather than rejecting the write. This is transparent to the writer but changes the schema contract for all readers. Consumers that read the column and explicitly cast it back to int may encounter overflow for values that exceed the int range. Consumers that infer schema from the Delta table metadata now see long where they previously saw int.

THE FIX

Monitor schema changes on critical Delta tables using Delta table history. Run `DESCRIBE HISTORY tablename` and check for operation type `CHANGE COLUMN` that indicates schema modifications. Set up alerts for schema changes on production tables, integrating with Microsoft Teams or your Azure Monitor alerting destination. For tables with strict type contracts, disable automatic type widening and handle type changes explicitly. Use `ALTER TABLE CHANGE COLUMN` for controlled type changes that allow downstream teams to update their consumer logic before the change is applied.

28

SCHEMA EVOLUTION

You enforced schema. A pipeline that was working for months suddenly failed.

THE SITUATION

Schema enforcement is enabled on a Delta table. A pipeline that has written to this table successfully for several months suddenly fails with a schema mismatch error. Investigation reveals that a recent source system change added a nullable column to the output that is not in the Delta table schema.

WRONG THINKING

Engineers enable schema enforcement to protect the table from unexpected schema changes. The enforcement is expected to prevent bad data from being written while allowing normal operations to continue. The possibility of a legitimate upstream change causing an enforcement failure is not anticipated when enforcement is first configured.

WHAT IS ACTUALLY HAPPENING

Schema enforcement in Delta Lake rejects any write where the source DataFrame contains columns not present in the table schema or where column types are incompatible. When a source system adds a new column, even a nullable one with no data, the write is rejected because the column does not exist in the Delta schema. Schema enforcement does not distinguish between intentional schema additions and erroneous ones. Any deviation from the defined schema causes a write failure regardless of whether the deviation is intentional.

THE FIX

Treat schema enforcement failures as signals for a schema governance process rather than as automatic rejections. When a pipeline fails due to schema enforcement, evaluate whether the source change is intentional and whether the Delta table schema should be updated. For intentional additions, update the Delta table schema explicitly using `ALTER TABLE ADD COLUMN` before allowing the pipeline to write. For monitoring, configure alerts when schema enforcement failures occur so they are addressed promptly rather than discovered after multiple pipeline failures.

29

SCHEMA EVOLUTION

Your schema evolution merged two incompatible schemas without warning.

THE SITUATION

Two pipelines write to the same Delta table. One pipeline evolves the schema by adding a column called status as a string. Another pipeline adds a column also called status but as an integer. Schema evolution accepts both writes. The status column in the table now contains mixed types and queries against it fail or return null for half the rows.

WRONG THINKING

Engineers assume schema evolution is safe when multiple pipelines write to the same table because Delta handles conflicts automatically. Schema conflicts between different writers are not anticipated because each writer manages its own schema independently.

WHAT IS ACTUALLY HAPPENING

When multiple writers independently evolve the same table schema, Delta applies schema evolution for each writer separately. If two writers add columns with the same name but different types, Delta may apply type widening if the types are compatible in the widening hierarchy, or may create ambiguity if they are not. In practice the second writer may successfully write with its type because Delta has already accepted the first writer's definition. The result is a column whose physical files contain values of different types written by different writers. Readers that expect a consistent type encounter null coercion errors or unexpected values.

THE FIX

For tables with multiple writers, establish a central schema governance process. Do not rely on each writer independently evolving the schema. Define the canonical table schema explicitly and require all writers to match it. Use a schema registry or a documented schema definition as the single source of truth, with Unity Catalog as the central metastore for type enforcement. Disable automatic schema evolution on multi-writer tables and require explicit schema changes through a controlled ALTER TABLE process that all writer teams are aware of before it takes effect.

30

SCHEMA EVOLUTION

You added a NOT NULL column to a Delta table. Existing rows became invalid.

THE SITUATION

A new column with a NOT NULL constraint is added to an existing Delta table that already contains millions of rows. The ALTER TABLE command completes successfully. Subsequent reads of the table fail for some consumers with null value errors. Existing rows that predate the column addition have null values for the new column because they were written before the column existed.

WRONG THINKING

Engineers add a NOT NULL constraint expecting it to enforce data quality going forward without affecting existing data. Since the ALTER TABLE succeeded, the table is assumed to be valid. The impact of the NOT NULL constraint on existing rows that predate the column is not considered.

WHAT IS ACTUALLY HAPPENING

Delta Lake allows adding columns with NOT NULL constraints through ALTER TABLE. However existing rows that were written before the column was added have null values for the new column because the column did not exist at write time. These rows violate the NOT NULL constraint but Delta does not validate existing data when the constraint is added. The constraint is enforced for new writes only. Consumers that read existing rows and enforce NOT NULL at the application layer encounter null values for the new column in pre-existing rows, which can cause null pointer exceptions or failed type casting.

THE FIX

When adding a NOT NULL column to an existing table, always provide a default value using ALTER TABLE ADD COLUMN with a DEFAULT clause so existing rows receive a non-null value. Alternatively update all existing rows to populate the new column before enforcing the NOT NULL constraint. Run a data quality check after adding the column to confirm no existing rows have null values for the new column before allowing downstream consumers to access the table. For tables where existing row validity is critical, test the ALTER TABLE change in a staging Azure Databricks workspace first.

31

SCHEMA EVOLUTION

Your Delta table schema shows 200 columns. Only 20 are actively used.

THE SITUATION

A Delta table has accumulated 200 columns over two years through repeated schema evolution events. Many of these columns were added for specific one-time analyses or deprecated features. Active queries use only 20 columns. Query performance is poor and the table schema is difficult to document and maintain.

WRONG THINKING

Engineers add columns as needed and rarely remove them because removing a column requires data migration and schema changes that feel risky. Unused columns are harmless from a correctness perspective so they are left in place. The cumulative performance and maintenance impact of 200 columns is not tracked.

WHAT IS ACTUALLY HAPPENING

Delta tables with many columns have larger Parquet file footprints because Parquet stores metadata for every column in every file. Reading a table with 200 columns when only 20 are needed still requires the reader to process column metadata for all 200. Column statistics in the transaction log are maintained for all columns, increasing log size. Queries that use `SELECT *` accidentally return 180 columns of null or irrelevant data. Schema documentation becomes a burden and new engineers cannot quickly identify which columns are authoritative.

THE FIX

Periodically audit column usage on wide Delta tables using Unity Catalog query history and system table analysis to identify columns that have not been referenced in any query in the past 90 days. For clearly unused columns, coordinate with data owners and downstream consumers before dropping them. Create a new version of the table with only the active columns using CTAS and replace the original if a column reduction is warranted. For ongoing governance, establish a policy that new columns added to shared tables must have a declared owner and a sunset date, preventing unbounded schema growth.

32

SCHEMA EVOLUTION

A column was renamed in the source. Schema evolution added a new column instead of renaming.

THE SITUATION

A source system renames an existing column from `customer_id` to `cust_id`. Schema evolution is enabled on the target Delta table. After the pipeline runs, the table contains both the old column `customer_id` with null values for new rows and a new column `cust_id` with values for new rows. Historical rows have `customer_id` populated and `cust_id` null. New rows have `cust_id` populated and `customer_id` null.

WRONG THINKING

Engineers assume schema evolution is intelligent enough to detect a column rename and map the old column name to the new one. Since the column contains the same data under a different name, the evolution should recognise this as a rename rather than a new column.

WHAT IS ACTUALLY HAPPENING

Delta Lake schema evolution has no concept of column rename detection. Schema evolution compares column names between the incoming DataFrame and the existing table schema. If a column name changes, Delta sees a new column that does not exist in the existing schema and an existing column that is no longer present in the source. It adds the new column name to the schema and marks the old column as having null values for new writes. There is no semantic understanding of whether the new column represents the same data under a different name. The result is a split history where the same logical data exists in two different column names depending on when the row was written.

THE FIX

Never rely on schema evolution to handle source column renames. When a source column is renamed, explicitly handle the rename in the pipeline transformation layer by mapping the new source column name back to the existing target column name using `withColumnRenamed()` or a column alias. This preserves the historical column name in the target table and maintains a consistent schema for all downstream consumers. If the target column name must also change, implement this as a controlled schema migration with explicit backfill of the renamed column for all historical rows.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 2

Chapter 5

Change Data Feed and Time Travel

Six scenarios covering the Azure storage overhead of Change Data Feed, the specific ways time travel queries return unexpected results, and what happens when retention settings do not match business or compliance requirements.

33

CHANGE DATA FEED

You enabled Change Data Feed. Storage costs increased significantly.

THE SITUATION

Change Data Feed is enabled on a Delta table on ADLS Gen2 to support downstream CDC consumers. After enabling CDF, monthly Azure storage costs for the table increase by 40 percent. The CDF data is consumed by one downstream pipeline that reads changes once per hour.

WRONG THINKING

Engineers enable CDF to support a downstream use case and assume the storage overhead is minimal because CDF stores only changes rather than full snapshots. The relationship between write frequency, CDF retention, and Azure storage cost is not calculated before enabling.

WHAT IS ACTUALLY HAPPENING

Change Data Feed stores a copy of every row that was inserted, updated, or deleted in dedicated CDF data files within the Delta table directory. For a table that receives frequent writes, CDF creates additional files equal to the volume of changed data for every write operation. These files are retained for the CDF retention period which defaults to the table's data retention window. A busy table with frequent small updates generates a continuous stream of CDF files that can equal or exceed the size of the table's active data. The 40 percent storage increase reflects the accumulated CDF files from all writes since CDF was enabled. On Azure, this also increases the storage account size against any cool tier or archive tier transition policies you may have configured, and it shows up directly on the Azure consumption bill.

THE FIX

Set an explicit CDF retention period that matches the actual consumption requirements rather than using the default table retention window. If the downstream consumer reads CDF data every hour, a 48-hour CDF retention window is more than sufficient. Configure the retention using `ALTER TABLE SET TBLPROPERTIES (delta.deletedFileRetentionDuration = 'interval 48 hours')`. Run `VACUUM` regularly to purge expired CDF files. Monitor CDF storage separately from table data storage in Azure Cost Management to track its contribution to total costs and adjust retention as consumption patterns change.

34

CHANGE DATA FEED

Your CDF query returns changes from 30 days ago by default.

THE SITUATION

A downstream pipeline reads CDF data from a Delta table using `readChangeFeed`. On the first run the pipeline processes changes correctly. On the second run after a restart the pipeline reprocesses all changes from 30 days ago rather than continuing from where it left off.

WRONG THINKING

Engineers implement a CDF consumer and test the first run successfully. Restart behavior is not tested before the pipeline goes to production. The assumption is that the pipeline will resume from its last position automatically similar to a streaming checkpoint.

WHAT IS ACTUALLY HAPPENING

Reading CDF data using `readChangeFeed` in batch mode requires explicitly specifying a starting version or timestamp. Without a starting version the query reads from the earliest available version in the CDF history. When the pipeline restarts, if no starting version is persisted and passed to the next run, the query starts from the beginning of the CDF history again, reprocessing all historical changes. Unlike Structured Streaming with checkpointing, batch CDF reads have no automatic position tracking. The consumer is responsible for persisting the last processed version and using it as the starting version for the next run.

THE FIX

Implement explicit version tracking for CDF consumers. After each successful batch read, persist the maximum version number from the CDF data to a control table or metadata store. On the next run, read the last processed version from the control table and use it as the `startingVersion` parameter in the `readChangeFeed` call. For streaming CDF consumption, use `spark.readStream.format('delta').option('readChangeFeed', 'true')` with checkpointing which handles position tracking automatically.

35

CHANGE DATA FEED

Change Data Feed is enabled. The `_change_type` column shows unexpected values.

THE SITUATION

A CDF consumer reads change records from a Delta table. Some records show a `_change_type` of `update_preimage` and `update_postimage` rather than simply `insert` or `update`. The consumer logic was written expecting only `insert`, `update`, and `delete` values. Records with `preimage` and `postimage` values are being processed incorrectly.

WRONG THINKING

Engineers implement CDF consumption expecting three change types: `insert`, `update`, and `delete`. The Delta Lake documentation for simple inserts and deletes is referenced but the update representation using `preimage` and `postimage` pairs is not fully understood before the consumer is built.

WHAT IS ACTUALLY HAPPENING

Delta Lake CDF represents row updates as two records per updated row: an `update_preimage` record containing the row values before the update and an `update_postimage` record containing the row values after the update. This allows consumers to see both the old and new state of each changed row. A consumer that treats `update_preimage` as an `insert` and `update_postimage` as an `insert` will insert duplicate records. A consumer that ignores `preimage` records will miss the before-state entirely. The full set of `_change_type` values is `insert`, `update_preimage`, `update_postimage`, and `delete`.

THE FIX

Update CDF consumer logic to handle all four `_change_type` values. For most upsert use cases, filter for `update_postimage` and `insert` records which represent the desired new state of each row. Discard `update_preimage` records unless the before-state is explicitly needed for audit or reconciliation purposes. Add a unit test that verifies correct handling of all four change types before deploying the consumer to production. Document the expected `_change_type` handling as part of the consumer specification.

36 CHANGE DATA FEED

You queried CDF for a specific version range. Some changes are missing.

THE SITUATION

A CDF consumer queries changes between version 100 and version 200 of a Delta table. The query returns fewer changes than expected. Investigation reveals that some updates that occurred within the version range are not appearing in the CDF output.

WRONG THINKING

Engineers assume a CDF query for a version range returns all changes that occurred within that range. The version numbers are assumed to be sequential with no gaps and CDF is assumed to capture every write operation.

WHAT IS ACTUALLY HAPPENING

CDF only captures changes from write operations that occurred after CDF was enabled on the table. If CDF was enabled at version 50 and the query starts at version 100, this is fine. However OPTIMIZE and other maintenance operations that rewrite data files also increment the Delta table version. If OPTIMIZE ran between version 100 and version 200, it increments the version counter without generating CDF change records because OPTIMIZE does not change the logical data content. The version range therefore includes versions where no logical data changes occurred, and the CDF output has gaps in version numbers that correspond to maintenance operations.

THE FIX

When consuming CDF by version range, account for version gaps caused by OPTIMIZE, VACUUM, and schema change operations. Use the `DESCRIBE HISTORY tablename` output to understand what operation corresponds to each version number before querying CDF ranges. For reliable change detection, use timestamps rather than version numbers as the CDF starting point when possible, as timestamps are more intuitive for defining change windows. Alternatively use streaming CDF consumption which automatically skips non-data versions.

37

CHANGE DATA FEED

Time travel query to yesterday works. Query to 31 days ago fails.

THE SITUATION

A data engineer runs a time travel query to inspect table state from yesterday and it succeeds. A compliance team requests a time travel query to 31 days ago and it fails with an error stating the version is no longer available.

WRONG THINKING

Engineers assume time travel is available for the full history of the table since the table was created. Since yesterday works, older versions are also assumed to be accessible. The retention configuration of the table is not checked before the compliance request is made.

WHAT IS ACTUALLY HAPPENING

Delta Lake time travel is only available for versions within the data retention window. The default retention window is 7 days. `VACUUM` removes data files for versions older than the retention threshold, making those versions unavailable for time travel. A table with default 7-day retention cannot serve time travel queries older than 7 days. If `VACUUM` has been running regularly with default settings, versions older than 7 days have had their underlying files deleted from ADLS Gen2. The transaction log still records the historical versions but the actual Parquet files they reference no longer exist.

THE FIX

Configure the Delta table retention window based on the longest time travel lookback required by business or compliance needs. Use `ALTER TABLE SET TBLPROPERTIES (delta.logRetentionDuration = 'interval 90 days', delta.deletedFileRetentionDuration = 'interval 90 days')` for 90-day retention. Be aware that longer retention windows increase ADLS Gen2 storage costs proportionally and may interact with lifecycle management rules on the storage account. For compliance use cases where specific historical states must be reproducible, consider writing explicit snapshots to a separate table at defined intervals rather than relying solely on time travel.

38

CHANGE DATA FEED

You restored a Delta table to a previous version. Downstream consumers broke.

THE SITUATION

A data quality issue is discovered in a Delta table. A restore operation rolls the table back to a previous version using `RESTORE TABLE tablename TO VERSION AS OF 50`. The table data is correct after the restore. Downstream streaming consumers that were reading from the table fail with version not found errors.

WRONG THINKING

Engineers restore the table to fix the data quality issue, which is the correct approach. The impact of the restore on downstream consumers that track their reading position by Delta version number is not considered before the restore is executed.

WHAT IS ACTUALLY HAPPENING

When a Delta table is restored to a previous version, new table versions are created that reflect the restored state, but the version numbers continue incrementing from the point of restore. Streaming consumers that use Delta as a source track their position using the last processed version number. After a restore, the version numbers the consumer has already processed no longer correspond to the data they read. If the consumer's last checkpoint was at version 80 and the table is restored to version 50 followed by new writes at versions 81 and above, the consumer may skip the restored data between versions 51 and 80 or encounter version conflicts depending on the streaming configuration.

THE FIX

Before restoring a Delta table, identify all streaming consumers using the table as a source and communicate the planned restore. After the restore, reset the checkpoints of all affected streaming consumers so they reprocess from the restored version rather than continuing from their pre-restore position. For critical production tables with active streaming consumers, test restore procedures in a staging Azure Databricks workspace first to understand the exact consumer impact. Document table restore as a breaking change event that requires coordinated consumer updates.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 2

Chapter 6

OPTIMIZE and VACUUM

Six scenarios covering the counterintuitive ways running maintenance too often, too aggressively, or in the wrong order causes problems rather than solving them on ADLS Gen2.

39

OPTIMIZE AND VACUUM

You ran VACUUM with 0 hours retention. Time travel stopped working.

THE SITUATION

VACUUM is run on a Delta table with the retention period set to 0 hours to immediately reclaim all ADLS Gen2 storage from deleted and old versions. Storage is successfully reclaimed. Immediately after, a business user attempts a time travel query and it fails. All historical versions are unavailable.

WRONG THINKING

Engineers run VACUUM with 0 hours retention to maximize storage savings on the Azure bill. The immediate storage reclaim is the goal. The impact on time travel capability is not considered because time travel was not an identified use case for this table.

WHAT IS ACTUALLY HAPPENING

VACUUM with 0 hours retention deletes all data files that are not part of the current table version. This includes all files that support time travel queries to any previous version. After running VACUUM with 0 hours, only the current version's data files remain on ADLS Gen2. Any time travel query that references a previous version fails because the underlying Parquet files for that version have been deleted. The transaction log still records historical versions but the files they reference no longer exist in the storage account.

THE FIX

Never run VACUUM with 0 hours retention on tables that have any time travel or CDF consumption requirements. The default retention of 7 days provides a reasonable safety window. For tables where storage cost is the primary concern, reduce retention to 2 to 3 days rather than 0. Before running VACUUM with reduced retention, confirm with all teams consuming the table that they do not require time travel beyond the new retention window. After running VACUUM with aggressive settings, add a note to the table documentation indicating the time travel window that is now available.

40

OPTIMIZE AND VACUUM

OPTIMIZE runs every hour. Your write performance degraded.

THE SITUATION

An OPTIMIZE job is scheduled to run every hour to maintain table compaction. Write performance to the same table degrades significantly over time. Investigation reveals that OPTIMIZE and the write pipelines are competing for resources and that OPTIMIZE is creating conflicts with concurrent write operations.

WRONG THINKING

Engineers schedule frequent OPTIMIZE runs to keep the table in optimal condition at all times. More frequent optimization seems safer than less frequent. The interaction between concurrent OPTIMIZE and write operations is not considered.

WHAT IS ACTUALLY HAPPENING

OPTIMIZE rewrites files in the target table by reading existing files, sorting and consolidating them, and writing new compacted files. This process modifies the table version just like any write operation. When OPTIMIZE and a write pipeline run concurrently, both read the current table state and attempt to commit at the same time. Delta's concurrency control detects that both operations modified overlapping files and forces one to retry. For busy tables with frequent writes, hourly OPTIMIZE runs create a continuous stream of conflicts that slow down write operations through repeated retries. OPTIMIZE also consumes cluster resources that compete with write pipelines on shared compute.

THE FIX

Schedule OPTIMIZE to run during low-write periods rather than on a fixed hourly cadence. For tables with continuous writes, run OPTIMIZE once per day after the main batch pipeline completes. Use `OPTIMIZE tablename WHERE partition_column = 'value'` to target only recently modified partitions rather than the full table, reducing conflict scope and execution time. On Databricks, enable Auto Optimize for tables with frequent small writes, which applies incremental compaction automatically with lower conflict risk than scheduled full-table OPTIMIZE.

41

OPTIMIZE AND VACUUM

You ran VACUUM. A concurrent read job failed with file not found.

THE SITUATION

VACUUM is run on a Delta table on ADLS Gen2 to clean up old files. A concurrent analytics query that was mid-execution when VACUUM ran fails with a file not found error from the abfss driver. The file that was deleted by VACUUM was still being read by the active query.

WRONG THINKING

Engineers run VACUUM as a maintenance operation believing it is safe to run at any time. VACUUM removes files that are no longer referenced by any current table version, so it seems harmless. The possibility that an active query is still reading a file that VACUUM considers eligible for deletion is not anticipated.

WHAT IS ACTUALLY HAPPENING

Delta Lake's VACUUM safety check uses the retention window to determine which files are safe to delete. Files not referenced by any version within the retention window are eligible for deletion. However an active long-running query that started before the VACUUM run may be reading files from an older version that falls outside the retention window. If those files are deleted from ADLS Gen2 by VACUUM while the query is still reading them, the query fails with a file not found error from the storage driver. Delta does not lock files that active queries are reading before running VACUUM.

THE FIX

Schedule VACUUM to run during maintenance windows when active long-running queries are not expected. For tables with long-running analytics queries, set the retention window to at least as long as the maximum expected query duration plus a safety buffer. Monitor active queries before running VACUUM and confirm no long-running queries against the target table are in progress. For critical production tables, implement a pre-VACUUM check that verifies no active sessions are reading the table before proceeding.

42

OPTIMIZE AND VACUUM

Your OPTIMIZE job runs for 6 hours on a table that receives 1000 rows daily.

THE SITUATION

A small Delta table receives 1000 rows per day from a pipeline. An OPTIMIZE job scheduled to run daily is taking 6 hours to complete. The table contains 2 years of data totalling about 700,000 rows. The OPTIMIZE job time is disproportionate to the data volume.

WRONG THINKING

Engineers run OPTIMIZE daily as a standard practice regardless of table size or write frequency. A 700,000-row table seems small and OPTIMIZE should be fast. The file count rather than the row count is not examined before investigating the slow OPTIMIZE.

WHAT IS ACTUALLY HAPPENING

A pipeline that writes 1000 rows per day using default Spark parallelism creates 200 files per day due to the default shuffle partition count. Over 2 years this produces 146,000 files for 700,000 rows. OPTIMIZE must read, sort, and rewrite 146,000 files to produce a compact result. The per-file overhead of listing, reading headers, and scheduling tasks for 146,000 files dominates the total runtime regardless of how few rows each file contains. On ADLS Gen2, each file open is also a separate authenticated request, so 146,000 reads cause the operation to spend most of its time in HTTP request round-trips rather than data processing. The table has a file count problem, not a data volume problem.

THE FIX

Fix the upstream write pipeline to produce fewer files per run before relying on OPTIMIZE to compensate. Use `coalesce()` or `repartition()` before writing to reduce the daily file creation from 200 to 1 to 5. Enable optimized writes in Databricks by setting `spark.databricks.delta.optimizeWrite.enabled = true` which automatically coalesces output files during writes. Once the upstream write is fixed, run a one-time OPTIMIZE to compact existing files, then the daily OPTIMIZE will only need to process a small number of new files rather than the full accumulated history.

43

OPTIMIZE AND VACUUM

VACUUM completed successfully. Storage costs did not decrease.

THE SITUATION

VACUUM is run on a Delta table with the default 7-day retention to reclaim storage from old file versions. VACUUM completes without errors. The Azure storage bill at the end of the month shows no reduction compared to previous months.

WRONG THINKING

Engineers run VACUUM expecting immediate storage cost reduction. Since VACUUM removes unreferenced files, storage should decrease. When the bill does not change, VACUUM is assumed to have not worked correctly.

WHAT IS ACTUALLY HAPPENING

VACUUM removes files that are outside the retention window and not referenced by any active table version. If the table has not been written to frequently or if most writes are appends rather than updates and deletes, there may be very few unreferenced files to remove. VACUUM is most effective on tables with frequent updates, deletes, or MERGE operations that create many superseded file versions. On an append-only table where files are never replaced, VACUUM has little to remove. Additionally, Azure storage billing is calculated based on average storage during the billing period, so a VACUUM run mid-month may not show a cost reduction until the following billing period. Storage tier transitions (hot to cool to archive) further complicate when a reduction appears on the bill.

THE FIX

Audit the table's file history before running VACUUM to estimate how much storage is actually eligible for reclamation. Use `DESCRIBE DETAIL tablename` to check the number of files and then compare with the raw storage listing in the Azure portal to estimate unreferenced file count. For append-only tables, VACUUM provides minimal benefit. Focus storage optimisation on write efficiency rather than cleanup. For tables with heavy update and delete activity, VACUUM provides the greatest benefit and should be run regularly. Use Azure Cost Management to track storage account costs over multiple billing periods rather than expecting an immediate reduction.

44

OPTIMIZE AND VACUUM

You disabled OPTIMIZE on a streaming table. File count grew to 500000.

THE SITUATION

A Structured Streaming job writes micro-batches to a Delta table every 30 seconds. OPTIMIZE was disabled because it conflicted with the streaming writes. After 3 months the table has 500,000 files and read performance has degraded severely. Enabling OPTIMIZE again causes it to conflict with streaming writes.

WRONG THINKING

Engineers disable OPTIMIZE after experiencing conflicts with the streaming job. Disabling OPTIMIZE seems like the logical solution to the conflict problem. The accumulation of files over time without any compaction is not anticipated as a larger problem than the occasional conflict.

WHAT IS ACTUALLY HAPPENING

Structured Streaming to Delta generates new files with every micro-batch. A job writing every 30 seconds generates 2 files per minute, 120 per hour, 2,880 per day. Without any compaction, file count grows indefinitely. At 500,000 files the transaction log is enormous, file listing operations are extremely slow, and every read must open and process metadata for hundreds of thousands of files. On ADLS Gen2 with hierarchical namespace, half a million files in a single table also brings the table close to per-storage-account request rate limits during any large read operation. The original conflict problem was an inconvenience. The uncontrolled file growth is a much more serious performance and operability issue.

THE FIX

Use the Databricks Auto Optimize feature which applies optimized writes and auto compaction specifically designed to work alongside streaming jobs without causing conflicts. Enable it with table properties `delta.autoOptimize.optimizeWrite = true` and `delta.autoOptimize.autoCompact = true`. Auto compaction uses a lower target file size than full OPTIMIZE and is designed to merge small files incrementally without conflicting with concurrent streaming writes. Alternatively run OPTIMIZE on a schedule that targets completed streaming micro-batches using partition-level optimization rather than full table OPTIMIZE.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 2

Chapter 7

Deletion Vectors and Row Tracking

Six scenarios covering deletion vector behavior, the compatibility issues they introduce with older Databricks Runtime readers and external connectors like Power BI and Fabric, and the specific ways row tracking fails to deliver the stable identifiers engineers expect.

45

DELETION VECTORS

You enabled deletion vectors. Your read performance dropped unexpectedly.

THE SITUATION

Deletion vectors are enabled on a Delta table to improve delete and update performance. Delete operations become nearly instantaneous as expected. However read performance for analytical queries against the table drops by 30 percent compared to the pre-deletion-vector baseline.

WRONG THINKING

Engineers enable deletion vectors expecting an improvement in both write and read performance. Deletion vectors are described as a performance improvement feature so they are expected to be universally beneficial. The read overhead of applying deletion vectors is not anticipated.

WHAT IS ACTUALLY HAPPENING

Deletion vectors store delete markers in a separate metadata file rather than rewriting the underlying Parquet files. This makes deletes fast because no data files need to be rewritten. However when reading data from a table with deletion vectors, the reader must load both the Parquet data files and the deletion vector files for each row group, then filter out the deleted rows at read time. For tables with many small deletes spread across many files, the reader must apply deletion vector filters across a large number of files. The read overhead of applying deletion vector filters can slow analytical queries, particularly full table scans, compared to reading compacted files where deleted rows have been physically removed.

THE FIX

Run `OPTIMIZE` periodically on tables with deletion vectors to physically remove deleted rows and consolidate deletion vector metadata into the rewritten files. After `OPTIMIZE` runs, the deletion vector overhead for the compacted files is eliminated. Set an `OPTIMIZE` schedule that balances delete write performance against read overhead. For tables where analytical read performance is more critical than delete write performance, do not enable deletion vectors or run `OPTIMIZE` frequently enough that deletion vector overhead stays minimal. Monitor read performance alongside write performance when evaluating deletion vector impact.

46

DELETION VECTORS

Deletion vectors are enabled. VACUUM is not removing deleted data.

THE SITUATION

Deletion vectors are enabled on a Delta table. Rows are deleted regularly. VACUUM is run on schedule. Azure storage costs are not decreasing as expected despite regular deletions and VACUUM runs.

WRONG THINKING

Engineers assume that deleting rows from a Delta table and running VACUUM will reclaim storage for the deleted data. The interaction between deletion vectors and VACUUM behavior is not understood.

WHAT IS ACTUALLY HAPPENING

Deletion vectors mark rows as deleted in a metadata file without modifying the underlying Parquet data files. The actual row data remains in the Parquet files on ADLS Gen2. VACUUM removes files that are no longer referenced by any active table version. When deletion vectors are used, the Parquet files that contain deleted rows are still referenced by the current table version because they contain other non-deleted rows. VACUUM cannot remove these files because they are still active. The deleted row data remains in the Parquet files even after VACUUM runs. Physical space is only reclaimed when OPTIMIZE rewrites the Parquet files, physically removing the deletion-vector-marked rows in the process.

THE FIX

Run OPTIMIZE after accumulating deletions to physically rewrite Parquet files and remove deletion-vector-marked rows from the physical storage. After OPTIMIZE rewrites the files, VACUUM can then remove the old pre-OPTIMIZE file versions that contained the deleted data. The correct sequence for storage reclamation with deletion vectors is OPTIMIZE followed by VACUUM, not VACUUM alone. Schedule OPTIMIZE to run before VACUUM in maintenance pipelines to ensure physical deletion and storage reclamation work together correctly.

47

DELETION VECTORS

You deleted rows with deletion vectors. The storage size did not change.

THE SITUATION

A large batch of rows is deleted from a Delta table that has deletion vectors enabled. The deletion completes almost instantly. Checking the table storage size on ADLS Gen2 immediately after shows no change in storage consumption.

WRONG THINKING

Engineers expect that deleting rows reduces storage immediately. Since the delete operation completed successfully and deletion vectors are supposed to be a performance improvement, the storage size should reflect the removed rows. The deferred storage reclamation model of deletion vectors is not understood.

WHAT IS ACTUALLY HAPPENING

Deletion vectors use a soft delete model. When rows are deleted, a small deletion vector file is written that records which rows in each Parquet file are deleted. The underlying Parquet files are unchanged. The logical table size decreases because the deleted rows are excluded from query results, but the physical storage size on ADLS Gen2 remains the same because the Parquet files still contain those rows on disk. Storage reclamation requires a subsequent OPTIMIZE run that rewrites the Parquet files without the deletion-vector-marked rows, followed by VACUUM to remove the old Parquet file versions.

THE FIX

Communicate clearly in pipeline documentation and Azure Cost Management dashboards that deletion vector-enabled tables require OPTIMIZE and VACUUM to reclaim physical storage from deleted rows. Implement a post-delete maintenance workflow that runs OPTIMIZE followed by VACUUM after large batch deletes. For storage cost monitoring, track both logical table size and physical storage size on ADLS Gen2 separately when deletion vectors are enabled, as the two will diverge between OPTIMIZE runs. Set a maximum acceptable divergence threshold and trigger OPTIMIZE automatically when the divergence exceeds it.

48

DELETION VECTORS

Your downstream Spark job cannot read a table with deletion vectors enabled.

THE SITUATION

Deletion vectors are enabled on a production Delta table. A downstream Spark job in a separate Azure Databricks workspace that reads this table and runs on an older cluster begins failing with an unsupported reader feature error. The job was reading the table without issues before deletion vectors were enabled.

WRONG THINKING

Engineers enable deletion vectors on a table without checking the Delta protocol version requirements. Since deletion vectors are a Databricks feature, all Databricks clusters are assumed to support them. The distinction between clusters running different Databricks Runtime versions and their supported Delta reader feature sets is not considered.

WHAT IS ACTUALLY HAPPENING

Deletion vectors are a Delta Lake table feature that upgrades the table protocol when enabled. Once a table has deletion vectors enabled, only clients that support the deletion vectors reader feature can read it. On Azure Databricks the minimum runtime that can read a deletion-vectors-enabled table is Databricks Runtime 12.2 LTS. Clusters running earlier runtime versions cannot read the table and produce an error along the lines of `requires reader table feature(s) that is unsupported by this version of Databricks: deletionVectors`. This breaks all downstream jobs on older clusters without any warning at the time deletion vectors are enabled. The issue also affects external consumers such as Power BI or Microsoft Fabric pipelines reading the table through their own Delta connectors if those connectors do not implement the deletion vectors reader feature.

THE FIX

Before enabling deletion vectors on shared production tables, audit all downstream consumers and their runtime versions. Any consumer on DBR 11.3 LTS or earlier cannot read the table after deletion vectors are enabled. Upgrade those consumers to DBR 12.2 LTS or later before enabling deletion vectors. For Power BI and Fabric consumers, verify their Delta connector versions implement the deletion vectors reader feature. Use the table's `DESCRIBE DETAIL` output to check the current reader and writer table features and understand what is already enabled. For tables with many downstream consumers on various runtime versions, coordinate the deletion vector enablement with a planned upgrade of all consumers rather than enabling on a fixed schedule. On DBR 14.1 and above you can also drop the deletion vectors table feature using `ALTER TABLE ... DROP FEATURE deletionVectors` if you need to roll back.

49

DELETION VECTORS

Row tracking is enabled. The row IDs are not stable across OPTIMIZE runs.

THE SITUATION

Row tracking is enabled on a Delta table to provide stable row identifiers for downstream processes that need to track individual row changes. After an OPTIMIZE run, downstream processes report that row IDs they had recorded are no longer valid and point to different rows.

WRONG THINKING

Engineers enable row tracking expecting it to provide permanently stable identifiers for each row. The assumption is that once a row is assigned an ID, that ID will always refer to the same row. OPTIMIZE is a maintenance operation that does not change data and therefore should not change row IDs.

WHAT IS ACTUALLY HAPPENING

Delta Lake row tracking assigns a unique row ID to each row that is preserved across updates and deletes. However OPTIMIZE rewrites data files by reading rows from existing files and writing them to new compacted files. During this rewrite, rows from different source files are merged and reordered. Row tracking preserves IDs for rows that came from the source files only if the implementation correctly propagates the existing row IDs through the rewrite operation. In some Delta and DBR version combinations, OPTIMIZE rewrites without fully preserving all row IDs, assigning new IDs to rows in the rewritten files. This is a known behavior in early row tracking implementations.

THE FIX

Verify the specific DBR version's row tracking stability guarantees before building downstream processes that depend on row ID permanence across OPTIMIZE runs. Check the Databricks release notes for the specific DBR version in use and confirm that row IDs are documented as stable across OPTIMIZE. For DBR versions where OPTIMIZE does not preserve row IDs, avoid using row IDs as permanent external references. Use business keys or application-assigned identifiers for stable cross-system references rather than relying on Delta-assigned row IDs for persistence across maintenance operations.

50

DELETION VECTORS

Your table has deletion vectors. A legacy reader is returning deleted rows.

THE SITUATION

A Delta table has deletion vectors enabled. A legacy application reads data from the table using an older Parquet reader that bypasses the Delta transaction log entirely, reading the Parquet files directly from ADLS Gen2 via abfss. This legacy reader returns all rows including rows that have been logically deleted via deletion vectors.

WRONG THINKING

Engineers assume that deleting rows from the Delta table removes them from all access paths. Since the deletion completed successfully and Delta queries correctly exclude deleted rows, the assumption is that deleted rows are universally inaccessible. The existence of applications that read raw Parquet files without going through the Delta reader is not considered.

WHAT IS ACTUALLY HAPPENING

Deletion vectors store delete markers in a separate metadata file. The underlying Parquet data files on ADLS Gen2 still contain the deleted rows physically. Applications that read the Parquet files directly via abfss, bypassing the Delta transaction log and deletion vector metadata, see the raw Parquet content including logically deleted rows. This is not a Delta bug but a data access pattern that bypasses Delta's abstraction layer. The same issue applies to direct Parquet access without Delta in any situation but deletion vectors make it more acute because deleted data persists in Parquet files indefinitely until OPTIMIZE physically removes it.

THE FIX

Identify and remediate all access paths to Delta tables that bypass the Delta reader. Applications should read Delta tables through the Delta reader interface, not by listing and reading raw Parquet files from ADLS Gen2. For legacy applications that cannot be updated to use Delta reader APIs, run OPTIMIZE frequently to minimize the window during which deleted rows persist in Parquet files. For regulatory requirements where deleted data must be absolutely inaccessible, ensure all access paths use Delta and run OPTIMIZE plus VACUUM immediately after compliance-required deletions. Use Unity Catalog access controls and storage-level ACLs to enforce that consumers can only reach the data through governed Delta access paths.

CONCLUSION

Delta Lake is deceptively simple. Until it is not.

Across 50 scenarios the pattern is consistent. Delta Lake works well at small scale and degrades silently as tables grow. File count accumulates on ADLS Gen2 without regular maintenance. MERGE performance degrades as target tables grow without Z-order. Schema evolution adds columns that invalidate statistics and break downstream Power BI consumers. CDF retention windows expire before consumers process the changes. Deletion vectors upgrade the table's reader protocol and break older clusters and external connectors. Azure storage costs creep up as small files, deletion vector metadata, and historical CDF data accumulate. None of these produce errors. They produce slow queries, wrong results, and Azure bills that nobody can explain.

ALSO IN THE SERIES

Clusters and Compute

Your job failed in 30 seconds. The cluster spent 28 of them starting up.

You enabled Spot VMs to cut cost. Jobs now randomly fail at 9am.

Workflows and Orchestration

Your workflow job shows success. One task silently produced no output.

Your DLT expectations are failing. The pipeline succeeds anyway.

And more. Download the full series free at ssanjaychandra.com

A NOTE ON THIS RELEASE

This is Book 2 in the Azure Databricks Engineering Patterns series. Every scenario is set in an Azure Databricks context: ADLS Gen2 with hierarchical namespace, abfss paths, Azure storage request rate limits, Power BI and Microsoft Fabric consumers, Unity Catalog as the central metastore, and the dual Azure VM and DBU billing model that surfaces in Delta storage and maintenance costs. Databricks and Azure both evolve quickly and some guidance may need updating as the platforms mature. If you spot a technical inaccuracy or a scenario that deserves deeper treatment, your feedback is genuinely welcome.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this book together.

CONNECT



www.ssanjaychandra.com



linkedin.com/in/ssanjaychandra



ssanjaychandra@outlook.com